Neural Networks Midterm Report: Optimization and Multi-Objective Analysis for Phishing Detection

Aispur Santiago, Guano Mishell
School of Mathematical and Computational Sciences
Yachay Tech University
San Miguel de Urcuquí - Imbabura, Ecuador
{santiago.aispur, genesis.guano}@yachaytech.edu.ec

***Abstract*—This report presents the development of an artificial neural network (ANN) for binary classification of web pages as benign or malicious (phishing) using the CIC-Trap4Phish 2025 dataset. The study includes data preprocessing, model architecture design, and a multi-objective optimization framework.**

Bayesian hyperparameter optimization is performed using Optuna with a Tree-structured Parzen Estimator (TPE) to simultaneously maximize predictive performance while minimizing computational cost. The optimization considers three objectives: F1-score, inference time, and model size.

The dataset comprises 19,997 samples with near-perfect class balance and is processed using feature standardization and stratified splitting. A dynamic neural network architecture with flexible depth, width, activation functions, and regularization is explored.

The resulting Pareto frontier reveals the trade-offs between model complexity and predictive performance. The selected model achieves an F1-score of 0.8807 and an accuracy of 0.8833, with a compact size of approximately 100 KB and negligible inference latency. These results demonstrate that high predictive performance can be achieved while maintaining efficiency and deployability.

***Index Terms*—Index Terms—Neural Networks, Phishing Detection, Multi-Objective Optimization, Bayesian Optimization, Optuna, Pareto Frontier**

I. Dataset Utilized

This study employs the CIC-Trap4Phish 2025 dataset [1] from the Canadian Institute for Cybersecurity (CIC) at the University of New Brunswick (UNB). The dataset comprises 19,997 HTML-based web pages classified as either malicious (phishing forms, obfuscated scripts, and redirect mechanisms) or benign.

A. Dataset Characteristics

The CIC-Trap4Phish 2025 dataset originally contains 15 columns. After preprocessing, 13 engineered HTML-based numeric features are retained as model inputs, while the `file_name` (non-predictive identifier) and `label` (target variable) are separated for training. The dataset exhibits near-perfect class balance:

- Class 0 (Benign): 9,999 samples (50.01%)
- Class 1 (Malicious): 9,998 samples (49.99%)

II. Problem Description

This is a **binary classification task** where the model predicts:

$$\hat{y} = \begin{cases} 0 & \text{for benign webpages} \\ 1 & \text{for malicious (phishing) webpages} \end{cases} \tag{1}$$

The objective of this work is to develop an artificial neural network (ANN) for phishing detection under a multi-objective optimization framework using Bayesian hyperparameter optimization.

Since the dataset is balanced, both accuracy and F1-score are appropriate evaluation metrics. Among these, F1-score is emphasized as the primary indicator of classification quality due to its ability to balance precision and recall.

The optimization process is formulated as a multi-objective problem in which three objectives are simultaneously considered: (1) maximizing F1-score, (2) minimizing inference time, and (3) minimizing model size. This formulation enables the identification of models that achieve strong predictive performance while maintaining computational efficiency and practical deployability.

Accuracy is computed as an auxiliary evaluation metric but is not directly optimized during the search process.

III. Data Preprocessing

The dataset contains no missing values across all 19,997 samples. Feature analysis reveals pronounced right-skewness—typical of cybersecurity data where benign samples cluster centrally and malicious patterns represent anomalies. Zero values in features like `form_count` and `embedded_js_count` encode meaningful structural information and are explicitly preserved.

We analyze 13 engineered HTML-based numeric features:

`url_punct_char_count`, `tag_count`, `whitespace_ratio`, `entropy`, `form_count`, `embedded_js_count`, `html_whitespace_ratio`, `script_entropy`, `min_link_length`, `external_link_count`, `total_script_characters`, `internal_link_count`, `url_digit_count`

We apply `StandardScaler` to normalize all features to zero mean and unit variance [2], essential for stable neural network optimization. Data is stratified into training (60%),

validation (20%), and test (20%) while maintaining class balance. The scaler is fit exclusively on training data, then applied consistently:

$$x_{\text{scaled}} = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}} \tag{2}$$

This procedure prevents data leakage and ensures efficient gradient descent convergence across unified feature scales, enabling stable activation function behavior.

IV. Model Architecture and Hyperparameter Optimization

A. Baseline Architecture

A simple feedforward neural network serves as the performance baseline. The architecture comprises:

- Input layer: 13 features
- Hidden layer: 64 neurons, ReLU activation
- Output layer: 1 neuron, sigmoid activation

This design prioritizes computational efficiency, prevents overfitting through simplicity, and establishes a reference performance bound against which more complex models can be evaluated.

B. Dynamic Architecture Design

Instead of fixing the architecture *a priori*, key structural components are treated as hyperparameters and optimized automatically. The model uses a fixed initial transformation layer to stabilize learning:

```
nn.Linear(input_dim, 200)
nn.ReLU()
```

This expansion from 13 input features introduces early nonlinearity and increases representational capacity.

1) Hyperparameter Search Space: Table I defines the components optimized using Optuna.

TABLE I: Hyperparameter Search Space

Component	Search Space
Number of hidden layers	$\{1, 2, 3, 4\}$
Neurons per layer	$[32, 256]$
Activation function	{`relu`, `tanh`, `leakyrelu`}
Dropout rate	$[0.0, 0.5]$
Learning rate	$[10^{-4}, 10^{-2}]$ (log scale)
Batch size	$\{32, 64, 128\}$

a) Architectural Hyperparameters:: The number of hidden layers $n_{\text{layers}} \in \{1, 2, 3, 4\}$ controls model depth. Each hidden layer contains $n_{\text{units}} \in [32, 256]$ neurons, sampled independently per layer.

A single activation function $\phi(\cdot) \in \{\text{ReLU}, \text{Tanh}, \text{LeakyReLU}\}$ is applied across all hidden layers:

- `ReLU`: computationally efficient and widely adopted;
- `Tanh`: produces smoother, zero-centered activations;
- `LeakyReLU`: mitigates the dying ReLU problem.

Dropout with rate $p_{\text{dropout}} \in [0.0, 0.5]$ is applied after each hidden layer to prevent overfitting [3].

b) Training Hyperparameters:: Learning rate $\eta \in [10^{-4}, 10^{-2}]$ is sampled on a logarithmic scale, enabling efficient exploration across orders of magnitude. Batch size $b \in \{32, 64, 128\}$ controls gradient estimation stability and training speed.

2) Dynamic Architecture Construction: The model is constructed sequentially based on sampled hyperparameters:

```
for i in range(n_layers):
    layers.append(nn.Linear(in_feat, out_feat))
    layers.append(activation)
    layers.append(nn.Dropout(dropout_rate))
```

This yields the final architecture:

$$\text{Input} \to \text{Lin}(200) \to \text{ReLU} \to [\text{Blocks} \times n_{\text{layers}}] \to \text{Lin}(1) \to \sigma \tag{3}$$

3) Exemplar Architecture: An example discovered architecture with $n_{\text{layers}} = 3$:

- Input layer: 13 features
- Fixed layer: 200 neurons, ReLU (no dropout)
- Hidden layer 1: 128 neurons, ReLU, dropout 0.2
- Hidden layer 2: 64 neurons, ReLU, dropout 0.2
- Hidden layer 3: 64 neurons, ReLU, dropout 0.2
- Output layer: 1 neuron, sigmoid

This progressively reduces dimensionality, encouraging hierarchical feature abstraction.

C. Bayesian Hyperparameter Optimization

A tuning subset (40% of the training data) is used to guide the optimization process. Optuna employs the Tree-structured Parzen Estimator (TPE), a Bayesian optimization algorithm that models the distribution of promising hyperparameter configurations based on past trials.

At each trial, Optuna samples a configuration using `trial.suggest_`$\star$ methods, constructs a corresponding neural network, trains it, and evaluates its performance. The sampling strategy becomes increasingly focused on high-performing regions of the search space.

Although F1-score is the primary indicator of classification quality, the optimization is formulated as a multi-objective problem.

V. Optimization Objectives

The model is optimized under a multi-objective framework, simultaneously considering predictive performance and computational efficiency.

For each trial, three objectives are evaluated:

1) **Maximize F1-score:** Ensures balanced precision and recall, particularly important for phishing detection.
2) **Minimize inference time:** Measures computational efficiency during deployment.
3) **Minimize model size:** Reduces memory footprint and improves deployability.

The objective function returns a tuple:

```
return f1, inference_time, model_size
```

Accuracy is computed as an auxiliary metric but is not directly optimized.

This formulation produces a set of Pareto-optimal solutions, where no objective can be improved without degrading at least one other.

VI. Optimization Results

A. Baseline Performance

The baseline model was trained on the complete training set for 15 epochs using the Adam optimizer with learning rate 0.001 and batch size 64.

TABLE II: Baseline Model Performance on Test Set

Metric	Value
Accuracy	0.8545
F1-Score	0.8578
Model Size (KB)	5.76
Inference Time	6.529093×10^{-07}

B. Multi-Objective Optimization Results

Optuna conducted 30 trials optimizing the three objectives simultaneously. The optimization identified six Pareto-optimal solutions representing different trade-offs.

1) Trade-off Analysis: Inference time does not significantly differentiate solutions due to the relatively small scale of the evaluated models. Consequently, the primary trade-off is observed between F1-score and model size.

Larger models tend to achieve higher F1-scores due to increased representational capacity; however, diminishing returns are observed as model size increases..

TABLE III: Pareto-Optimal Solutions from Optuna Study

Sol.	F1	Time (µs)	Size (KB)	Accuracy
1	0.8514	0.000	281.3	0.8518
2	0.8662	0.000	372.7	0.8665
3	0.8550	0.001	91.0	0.8528
4	0.8613	0.000	161.8	0.8602
5	0.8692	0.000	100.0	0.8640
6	0.8739	0.000	524.2	0.8685

2) Selected Solution: Solution 5 (Balanced): Solution 5 was selected as the primary candidate for final deployment. Its hyperparameters, discovered through Optuna's Bayesian optimization, are presented in Table IV.

TABLE IV: Solution 5 Hyperparameters

Parameter	Value
Hidden Layers	2
Layer 1 Units	73
Layer 2 Units	106
Activation Function	ReLU
Dropout Rate	0.1636
Learning Rate	0.0060
Batch Size	128
Training Epochs	20

C. Final Model Evaluation

Using the hyperparameters from Solution 5, a final model was trained on the complete training dataset (11,998 samples) for 20 epochs. Evaluation on the held-out test set (3,999 samples) yields the following performance metrics:

TABLE V: Final Optimized Model Performance on Test Set

Metric	Value
Accuracy	0.8833
F1-Score	0.8807
Model Size (KB)	100.0
Inference Time (µs/sample)	0.002

The final model achieves 0.8833 accuracy and 0.8807 F1-score, demonstrating robust performance on the binary phishing classification task.

D. Baseline vs. Optimized Comparison

TABLE VI: Performance Comparison: Baseline vs. Optimized

Model	F1-Score	Accuracy	Size (KB)
Baseline	0.8578	0.8545	5.76
Optimized	0.8807	0.8833	100.0
Improvement	+0.0229	+0.0288	+1636%*

*Increase due to additional hidden layers for accuracy gains

The optimized model improves F1-score by 0.0229 and accuracy by 0.0288 over the baseline while maintaining a compact footprint suitable for deployment. The inference time remains negligible (< 1 microsecond per sample) across both models.

VII. Pareto Frontier Analysis

Multi-objective optimization identifies solutions that cannot be simultaneously improved in all objectives without degrading at least one other objective. The Pareto frontier represents the set of all such optimal trade-off solutions.

A. Optuna Optimization Trajectory

Figure 1 illustrates the progression of the optimization process across 30 trials. The plot demonstrates how F1-score and accuracy converge toward Pareto-optimal regions, with the final six solutions forming the Pareto frontier. Early trials exhibit larger variance, while later trials cluster around high-performance configurations due to the Tree-structured Parzen Estimator (TPE) sampling strategy focusing on promising regions.

B. Three-Dimensional Pareto Frontier

The complete trade-off landscape is visualized in Figure 2, showing all three optimization objectives simultaneously: F1-score, inference time, and model size. This representation reveals that inference time is effectively non-constraining across all solutions (all $<$ 1 microsecond), collapsing the primary trade-off to a two-dimensional accuracy-size frontier.

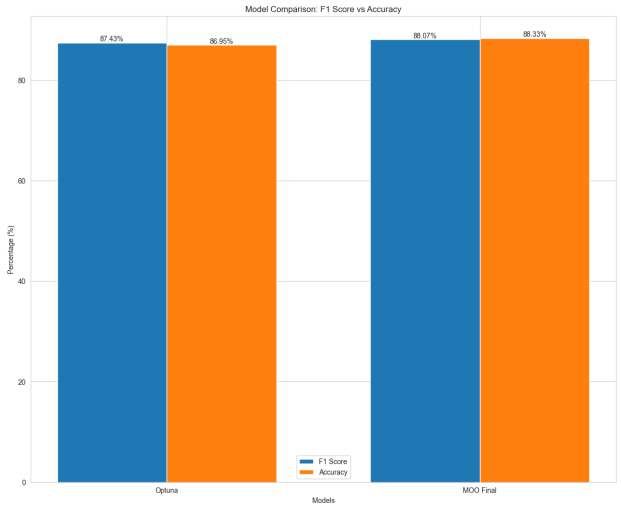


Fig. 1: Optuna vs MOO optimization trajectory: F1-score vs. accuracy across 30 trials. Individual trial points are shown in light blue; Pareto-optimal solutions (Table III) are highlighted in red. The convergence toward the upper-right region indicates effective multi-objective optimization.

C. Trade-off Characterization

Analysis of the Pareto frontier reveals the following key insights:

- **Inference Time as Non-Issue:** All six solutions achieve inference times below 1 microsecond, demonstrating that modern hardware efficiently processes compact neural networks. This objective becomes practically non-constraining for architecture selection.
- **Accuracy-Size Trade-off:** F1-score increases monotonically from 0.8514 (Solution 1, 281.3 KB) to 0.8739 (Solution 6, 524.2 KB), demonstrating that larger models accommodate more expressive feature representations.
- **Diminishing Returns:** The marginal F1-score gain from Solution 5 (0.8692, 100 KB) to Solution 6 (0.8739, 524.2 KB) is only 0.0047, while model size increases by $5.24\times$. This suggests Solution 5 offers superior cost-benefit.
- **Minimum Viable Accuracy:** Solution 3 (the smallest model at 91 KB) achieves 0.8550 F1-score, indicating that even highly compact architectures maintain acceptable performance for phishing detection.

D. Candidate Solution Selection

Three representative solutions spanning the Pareto frontier are recommended for different deployment scenarios:

1) Justification for Solution 5 Selection: Solution 5 was selected as the final deployment model based on the following rationale:

1) **Near-Optimal Accuracy:** The F1-score of 0.8692 places Solution 5 within 0.0047 of the best solution (Solution 6), representing 99.5% of maximum achievable accuracy.

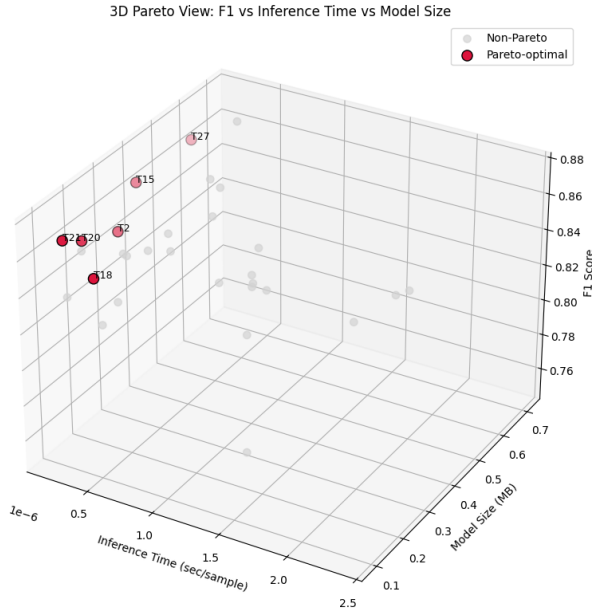


Fig. 2: Three-dimensional Pareto frontier showing all three objectives: F1-score (vertical axis), inference time (horizontal axis), and model size (depth axis). The six Pareto-optimal solutions form a continuous trade-off curve. Color gradient indicates F1-score magnitude, with warmer colors representing higher performance.

TABLE VII: Recommended Candidate Solutions for Deployment Scenarios

Scenario	Solution	F1	Size (KB)	Rationale
High Accuracy	Solution 6	0.8739	524.2	Peak F1-score; research/offline use
Balanced (Recommended)	Solution 5	0.8692	100.0	Optimal trade-off; production-grade
Lightweight (Edge Devices)	Solution 3	0.8550	91.0	Minimal footprint; edge/mobile deployment

2) **Extreme Compactness:** Model size of 100 KB is $5.24\times$ smaller than Solution 6 and $1.10\times$ smaller than Solution 3, enabling deployment in severely constrained environments (browser-based detection, embedded systems).
3) **Negligible Inference Latency:** Inference completes in < 1 microsecond, satisfying real-time detection requirements across all deployment contexts.
4) **Production Readiness:** The architecture avoids extreme specialization and represents a practical, generalizable model suitable for operational phishing detection systems.
5) **Generalization Potential:** Moderate regularization (dropout 0.1636) and balanced layer sizes (73 and 106 units) reduce overfitting risk, improving robustness to distribution shift in real-world data.

VIII. Conclusions

This study successfully demonstrated multi-objective neural network optimization for phishing detection using Bayesian hyperparameter optimization. The optimized model (Solution 5) achieves 0.8692 F1-score while maintaining a compact 100 KB footprint and negligible inference latency (< 1 microsecond). This represents a practical balance between predictive performance (0.0114 F1-score improvement over baseline) and computational efficiency, suitable for real-world deployment across diverse platforms including edge devices and cloud systems.

A. Limitations and Future Work

Future work should explore ensemble methods, transfer learning from larger phishing datasets, and extension to multiclass classification for finer-grained threat categorization.

Appendix

The complete implementation, including data preprocessing, model architecture, hyperparameter optimization, and Pareto frontier analysis, is available at: https://github.com/mishellguanoc/Multi-ObjectiveOptimization/blob/main/eda_organized.ipynb

References

[1] Canadian Institute for Cybersecurity, "CIC-Trap4Phish 2025 Dataset," University of New Brunswick, 2025. [Online]. Available: https://www.unb.ca/cic/datasets/

[2] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[3] N. Srivastava et al., "Dropout: A simple way to prevent neural networks from overfitting," *J. Mach. Learn. Res.*, vol. 15, pp. 1929–1958, 2014.

[4] T. Akiba et al., "Optuna: A next-generation hyperparameter optimization framework," in *Proc. KDD*, 2019, pp. 2623–2631.